REPÚBLICA PORTUGUESA
EDUCAÇÃO, CIÊNCIA E INOVAÇÃO

AEGP
AGRUPAMENTO DE ESCOLAS GABRIEL PEREIRA - ÉVORA

AGRUPAMENTO DE ESCOLAS GABRIEL PEREIRA - ÉVORA

Programação e Sistemas de Informação

MÓDULO 3

PROGRAMAÇÃO ESTRUTURADA

CURSO PROFISSIONAL

TÉCNICO DE GESTÃO E PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS

PROGRAMAÇÃO E SISTEMAS DE INFORMAÇÃO

Programação Estruturada

- AMBIENTE DE DESENVOLVIMENTO *VISUAL STUDIO*
- SINTAXE DA LINGUAGEM C#

Sintaxe da Linguagem C#

Variáveis e tipos de dados

Existem diferentes tipos de variáveis (definida com diferentes palavras reservadas), como por exemplo:

int ou **long** - inteiros (números inteiros), sem decimais, tais como 123 ou -123

double ou **float**- números com casas decimais, tais como 19,99 ou -19,99

char- caracteres simples, como 'a' ou 'B'. valores Char ficam entre aspas simples

string- texto, como "Olá Mundo". valores string ficam entre aspas duplas

bool - valores com dois estados: verdadeiro ou falso

```
string name = "John";
Console.WriteLine(name);

int myNum = 15;
Console.WriteLine(myNum);
```

Nota: se atribuir um **novo valor** a uma variável existente, o valor anterior vai ser **substituído**

```
int myNum = 15;
myNum = 20;
// myNum é agora 20
Console.WriteLine(myNum);
```

Sintaxe da Linguagem C#

Variáveis

Devem ser identificadas com nomes exclusivos, que podem ser nomes curtos (como x e y) ou nomes mais descritivos (idade, soma, totalVolume).

Regras gerais para a construção de nomes para as variáveis (identificadores únicos):

- Podem conter letras, números e o caracter underscore (_);
- Devem começar com uma letra minúscula e não podem conter espaços em branco
- "myVar" e "myvar" são variáveis diferentes (sensível às maiúsculas);

Palavras reservadas (tais como **int** ou **double**) **não podem** ser usadas como nomes.

NOTA

Devem ser usados **nomes que identifiquem o conteúdo**, a fim de criar um **código compreensível** e de **fácil manutenção**.

Sintaxe da Linguagem C#

WriteLine() com diferentes tipos de variáveis

```
string name = "John";
Console.WriteLine("Hello " + name);
```

```
int x = 5;
int y = 6;
Console.WriteLine(x + y);
```

```
string firstName = "John ";
string lastName = "Doe";
string fullName = firstName +
lastName;
Console.WriteLine(fullName);
```

```
int x = 5, y = 6, z = 50;
Console.WriteLine(x + y + z);
```

Sintaxe da Linguagem C#

Constantes

Quando queremos armazenar **um valor que não se altera ao longo da execução do programa**.

Exemplo muitas vezes referido como uma constante: **PI** (3,14159 …).

Utiliza-se a palavra reservada **const.**

```
const int num = 15;
```

Sintaxe da Linguagem C#

Tipos de Dados

Tipo	Tamanho	Descrição
int	4 bytes	Armazena números inteiros de -2 147 483 648 a 2 147 483 647
long	8 bytes	Armazena números inteiros de -9 223 372 036 854 775 808 a 9 223 372 036 854 775 807
float	4 bytes	Armazena números fracionários. Suficiente para armazenar 6 a 7 dígitos decimais
double	8 bytes	Armazena números fracionários. Suficiente para armazenar 15 dígitos decimais
bool	1 bit	Armazena valores verdadeiros ou falsos
char	2 bytes	Armazena um único caracter/letra
string	2 bytes por caracter	Armazena uma sequência de caracteres

Sintaxe da Linguagem C#

Tipos de Dados Numéricos

int - é o tipo de dados preferido quando criamos variáveis com um valor numérico.

```
int myNum = 100000;
Console.WriteLine(myNum);
```

long - utilizado quando o tipo **int** não é grande o suficiente. Deve terminar com um "L“.

```
long myNum = 15000000000L;
Console.WriteLine(myNum);
```

float - deve terminar o valor com um "F“.

```
float myNum = 5.75F;
Console.WriteLine(myNum);
```

double – quando o tipo **float** não é suficiente.

```
double myNum = 19.99;
Console.WriteLine(myNum);
```

Sintaxe da Linguagem C#

float ou **double**?

A precisão de um valor com virgula flutuante indica quantos dígitos o valor pode ter depois do ponto decimal. A precisão do **float** é de apenas **6 ou 7 dígitos decimais**, enquanto que no caso do **double** as variáveis têm uma precisão de cerca de **15 dígitos**.

Por isto, é mais seguro usar double para a maioria dos cálculos.

Podem também ser usados para números em notação científica, com um "e" para indicar a potência de 10:

```
float f1 = 35e3F;
double d1 = 12E4;
Console.WriteLine(f1);
Console.WriteLine(d1);
```

String

Mais que um tipo de dados, é um **objeto**, que contem propriedades e métodos que podem ser executados em certas operações.

Length – comprimento da string

```
string txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
Console.WriteLine("O tamanho da string é : " + txt.Length);
```

ToUpper() e ToLower() – maiúsculas e minúsculas

```
string txt = "Hello World";
Console.WriteLine(txt.ToUpper());
Console.WriteLine(txt.ToLower());
```

Sintaxe da Linguagem C#

String

Junção (ou concatenação) de variáveis do tipo String

```
string firstName = "John";
string lastName = "Doe";
string name = firstName + " " + lastName;
Console.WriteLine(name); //o mesmo que Console.WriteLine("John" + " " + "Doe“);
```

Booleano

A expressão booleana retorna um **valor do tipo booleano**: True ou False

```
bool a = true;
bool b = false;
Console.WriteLine(a);
Console.WriteLine(b);
```

Podemos usar um operador de comparação, como o maior (>), que retorna um valor booleano

```
int x = 10;
int y = 9;
Console.WriteLine(x > y);
```